ROADMAP — Aula, Phases 7+

Companion to `docs/PRD.md`. Phases 0–5 are shipped. Phase 6 (multi-tenancy / monetization) stays parked on purpose.

This document covers the next block of work. Its organizing premise:

> Phases 0–5 built a **written output correction engine**. That is one skill of four, and — per the SLA literature — not the one that drives most acquisition. Phases 7–10 turn Aula from a grading tool into a learning loop: elicit the structures I'm failing, make me revise, focus the feedback, and add an input channel.

Phase 6.5 — Documentation sync (do first, 20 minutes)

`docs/PRD.md` and `CLAUDE.md` both still describe Phase 2 as "next." Phases 2–5 are shipped. Claude Code reads these as ground truth, so stale docs actively misdirect implementation.

Tasks

- Update the `§7 Roadmap` table in `PRD.md`: mark 2–5 Done, add Phases 7–10.
- Update `CLAUDE.md`'s "What we're building" paragraph to state current phase.
- Add a `## Hard rules` entry: **trend-affecting schema is append-only.** New columns may be added to `error_observations`; existing columns and the 25-category taxonomy are never renamed or repurposed without a migration and a backfill plan. Trend comparability from entry #1 is load-bearing.

Phase 7 — Targeted prompt elicitation

The problem this solves. Prompts currently rotate by *topic*. Accuracy is measured as correct ÷ obligatory contexts — but obligatory contexts only appear by chance. That means the denominator is not under my control, weak categories can go weeks without data, and the avoidance flag can't distinguish "he's avoiding subjunctive" from "no prompt this fortnight happened to require it."

The fix. Seed the prompt generator from the error log so prompts *structurally force* the contexts I'm failing. This is elicited-production methodology applied directly: if `subjunctive_trigger`

is flagged, generate a prompt that cannot be answered well without it ("Escribe lo que le recomendarías a un amigo que quiere invertir por primera vez…").

Scope

- Reuse the existing read-time weak-category query (escalation flags first, then lowest-accuracy categories with ≥5 in-window obligatory contexts).
- Select **1–2 target categories per prompt.** Never more — a prompt engineered for four structures reads like a grammar exercise and kills the writing.
- Add an **elicitation template library**: for each of the 25 categories, 2–4 prompt *shapes* known to force that structure. Plain data, checked into `/src/shared/prompts/elicitation/`. This is a linguistics artifact, not generated content — write it deliberately, don't have a model invent it.
- Compose: topic (existing rotation) × elicitation shape (new) → Haiku pass to make the result read naturally in Mexican Spanish. Cache the system prefix.
- **Covert by default.** Do not tell me which category is being targeted. If I know the prompt is fishing for subjunctive I will monitor for it, and the resulting accuracy number measures my monitoring, not my acquired competence. Ship a `Mostrar objetivo` toggle, default off, for when I want to drill consciously — but store which mode was used.

Schema

ALTER TABLE entries

ADD COLUMN target_categories text[] NULL, -- categories the prompt elicited

ADD COLUMN elicitation_mode text NULL; -- 'covert' | 'disclosed' | NULL (legacy/freeform)

Nullable, so every existing Phase 1–5 entry stays valid and queryable.

Why this is worth doing first

It's the cheapest item on this list — a template library and a prompt-composition change, no new tab, no new grading contract — and it makes every metric already built work better. It also unlocks a real analysis later: accuracy on *elicited* contexts vs. *spontaneous* ones is a direct measure of the gap between monitored and automatic knowledge.

Definition of done

- A category with an active escalation flag is selected as a target on the next generated prompt.

- Across 5 real entries written to elicited prompts, the target category shows ≥3 obligatory contexts per entry in the grader's `category_summary`. If it doesn't, the template for that category is wrong — fix the template, that's the actual deliverable being tested.
- `target_categories` and `elicitation_mode` persist and are visible in History.
- Freeform journal entries still work with both columns NULL.
- No change to the grading contract JSON.

Non-goals

Targeting Workbook or Flashcards from the same signal (already partly exists, leave it). Multi-category prompts. Any change to how accuracy is computed.

Phase 8 — The revision cycle

The problem this solves. Feedback is currently terminal. I read corrections and move on. The written-corrective-feedback research (Bitchener & Knoch, Ferris, Storch) converges on the point that uptake happens during *revision* — the moment of re-production — not at the moment of reading a correction. Aula has no re-production step, so most of what the grader produces is never acted on.

This also supersedes the open "entry regrade" item: the reason to resubmit an entry stops being "the grading broke" and becomes "I'm fixing it on purpose."

Scope

Flow: entry → feedback → `Revisar` → revision editor → second grading scored on uptake → both versions retained and viewable side by side.

Indirect feedback in the revision editor. This is the important design decision. The revision view shows *where* the errors were and *what category* each one is — it does **not** show the correction. Handing me the corrected text to copy out teaches nothing; forcing retrieval is the whole mechanism. The full corrected text stays available behind an explicit "ver correcciones" reveal, which is logged (see below).

Uptake scoring. The second grading pass gets the parent entry's observation list as input and returns, per flagged error: fixed / still wrong / newly wrong. New metric: **uptake rate** = errors corrected in revision ÷ errors flagged, per category, per 14-day window.

Schema

ALTER TABLE entries

```
  ADD COLUMN parent_entry_id uuid NULL REFERENCES entries(id),

  ADD COLUMN revision_number int NOT NULL DEFAULT 0,

  ADD COLUMN revealed_corrections boolean NOT NULL DEFAULT false;

ALTER TABLE error_observations

  ADD COLUMN is_revision boolean NOT NULL DEFAULT false,

  ADD COLUMN resolves_observation_id uuid NULL REFERENCES error_observations(id);
```

Critical correctness rule

Revision observations are excluded from accuracy and exposure trends by default. A revision is a second attempt at the same obligatory contexts with the answers half-given; counting it inflates accuracy and double-counts exposure, which would corrupt every trend line built so far. Trends filter `is_revision = false`. Uptake is its own separate series.

Get this wrong and the damage is silent and retroactive. Write the test first.

Definition of done

- A revision saves with correct `parent_entry_id` and `revision_number = 1`.
- History accuracy/exposure trends are numerically **identical** before and after a revision is saved. Test this explicitly with a fixture.
- Uptake rate displays per category once ≥5 flagged errors exist in window.
- Revision editor shows error location + category, never the correction, until the reveal is tapped; `revealed_corrections` records it.
- A second revision (`revision_number = 2`) of the same entry works.

Phase 9 — Focused feedback budget (small, do alongside 7)

The problem this solves. The grader explains every error. That's *unfocused* corrective feedback, and the evidence for it is thin beyond surface accuracy — attention is finite, and fourteen equally-weighted explanations dilute all fourteen. Focused CF on a small number of targets outperforms it fairly consistently.

Scope

- **The grading contract JSON does not change.** Every error is still detected, tagged, and corrected in `corrected_text`. This is non-negotiable — the observation stream is what all trends are built from.
- What changes is the **explanation budget**. The grader receives a `focus_categories` parameter (the Phase 7 target categories, or the current escalation flags for freeform entries) and is instructed to write a full "why this rule works this way" note for those, and a terse one-line note for everything else.
- Same for `feedback_prose`: the debrief addresses the focus categories in depth and lists the rest.
- UI: focused errors render expanded, the rest collapse behind a count.

Definition of done

- Two gradings of the same fixture entry with different `focus_categories` produce the same observation count and the same `category_summary`, with different note depth.
- Measured output-token reduction on a long entry, read from `usage_log`.
- The truncation guard added on 07-10 still passes — shorter output should make this safer, not riskier, but verify.

Phase 10 — Lectura (the input tab)

The problem this solves. Aula has no input channel. Correcting output is remediation; comprehensible input at volume is what builds the implicit system being remediated. This is the largest single gap in the app.

The interesting part isn't glossing — it's **input enhancement** (Sharwood Smith): surfacing, inside authentic text, the exact structures I'm currently failing, so reading doubles as targeted noticing practice on the same categories the error log is tracking.

Scope

v1 input: pasted text only. No URL fetching, no scraping, no paywalls. Paste from BBC Mundo, Bloomberg Línea, El País, whatever Flavia sends.

Pipeline:

1. **Tokenize + frequency-profile** — plain code, no model. Rank every token against a Spanish frequency list (top 5000, checked into the repo).

2. **Known-item cross-reference** — plain code. Diff against the `known_cards` ledger and `word_bank`. Anything known is not glossed. This is what makes the glossing *mine* rather than generic.
3. **Gloss the remainder** — Haiku, one batched call for the whole text, glossing in context (never bare dictionary entries — the Portuguese-interference rule from `CLAUDE.md` applies here more than anywhere).
4. **Input enhancement** — highlight instances in the text where a currently- flagged category appears *correctly used by a native writer*, with a one-line noticing prompt on tap. Toggleable; off by default so I can read normally.
5. **Comprehension questions** — 4–6, Haiku, in Spanish, at level.
6. **Capture** — one-tap to Word Bank with the real context sentence and the real capture date. Reuse the date-threading fix from 07-11; never let a model invent a `leccion::` date.

Hard rule

Lectura never writes to `error_observations`. Reading comprehension is not a production error. Input is input. If comprehension tracking is wanted later it gets its own table.

Schema

```
CREATE TABLE input_texts (

  id uuid PRIMARY KEY,

  title text,

  source text,              -- free text: 'BBC Mundo', 'Flavia', etc.

  body text NOT NULL,

  word_count int,

  unknown_token_count int,

  created_at timestamptz DEFAULT now()

);

CREATE TABLE input_lexis (

  id uuid PRIMARY KEY,

  input_text_id uuid REFERENCES input_texts(id) ON DELETE CASCADE,
```

```
  term text NOT NULL,
  lemma text,
  frequency_rank int,
  context_sentence text,
  gloss text,
  known_at_encounter boolean,
  captured_to_word_bank boolean DEFAULT false
);
```

Model routing

Haiku for glossing and questions, batched into as few calls as possible. Tokenizing, ranking, and known-item diffing are plain code — do not spend a model call on anything deterministic.

Definition of done

- A pasted 600-word article renders with unknown items glossed and known items untouched, verified against a real `known_cards` state.
- Comprehension questions generate in Spanish at the stored DELE level.
- Enhancement toggle highlights a currently-flagged category and can be turned off.
- One-tap capture writes to `word_bank` with correct context sentence and real date.
- `error_observations` row count is unchanged by any Lectura activity.
- Total model cost for one article is visible in `usage_log`.

Backlog (specced when Phase 10 lands, not before)

Spaced/interleaved Workbook scheduling. Workbook generates on demand; retrieval-practice research says the *schedule* matters more than the item. Give each error category lightweight review state (last drilled, interval, stability) and have Workbook propose a daily mix interleaved across 3–4 categories rather than blocked on one. Reuse the FSRS concept, not the parser code.

Lexical profiling — no API cost. Run the full corpus of my entries against the frequency list: % of tokens outside top-1000/2000/5000, type-token ratio, verb-form variety over time. Objective productive-vocabulary curve, catches *lexical* avoidance that the grammar taxonomy structurally cannot see. Plain Python. Strong portfolio detail — it's real computational linguistics rather than another model call.

DELE task-format mode. Timed, exam-shaped writing tasks graded against published DELE band descriptors instead of the internal sophistication score. This is what converts general improvement into a pass.

Speaking ingest. Record a voice memo → transcribe → run the transcript through the *same* grader with `source = 'speaking'`. Same taxonomy, same trend math, excluded from written trends but comparable to them. The written/spoken accuracy gap on identical categories is where the real B2 risk lives.

`known_structures` — still documented, still unwired. Needs a settings column, a Settings UI, and threading into Workbook and Flashcards prompts. Without it, "level-appropriate" means "what the CEFR says an A2 knows" rather than "what Flavia has actually taught me." Should ride along with Phase 7, since the elicitation templates need it: don't elicit a structure I've never been taught.

Mobile keyboard check in Lessons — needs a real phone, not a simulator.

Persona document — `Persona.pdf` still describes Buenos Aires / Panama City, A1–A2, a six-month B2 timeline, and daily one-hour guides. None of that is current. It is live context in every conversation in the project and pulls against the Mexico City / DELE direction. Rewrite or retire it.

Suggested sequencing for this week

Day	Work
1	Phase 6.5 doc sync. Phase 7 schema migration (backed up, versioned, tested against a copy of real data).
1–2	Phase 7 elicitation template library — this is the slow, careful part. Write the templates by hand.

Day	Work
2–3	Phase 7 prompt composition + covert/disclosed toggle. Ship. Write entries against it.
3	Phase 9 focused feedback budget (small — one prompt change plus UI collapse).
4–5	Phase 8 revision cycle. Write the trend-isolation test **before** the feature.
5	Phase 10 spec review only. Do not start building Lectura until 7 and 8 have produced a week of real entries.

Phases 7 and 8 change what I do daily. Phase 10 adds something I don't do at all. Get the first two working and generating real data before adding surface area — same reason v1 was scoped to Writing alone.

Standing rules for this block of work

1. **Migration safety** (PRD §8.5) applies to every schema change here: back up, versioned migration file, tested against a copy of real production data.
2. **Every new column is nullable or defaulted** so all existing entries and observations remain valid and queryable.
3. **The grading contract JSON shape does not change** in Phases 7–10.
4. **Anything deterministic gets no model call.** Frequency ranking, known-item diffing, answer matching, tokenizing — plain code.
5. **Haiku for volume, Sonnet for judgment.** Batch aggressively; cache the stable system + taxonomy + rubric prefix.
6. **Write the contract down before building** — the Flashcards rebuild on 07-10 is the proof. `ANKI_SCHEMA.md` turned a plausible-looking useless feature into one that imported cleanly on the first real try. Do the same for the elicitation template library and the uptake-scoring contract.